Informe de Laboratorio: Estructura de Computadores

Nombre de los estudiantes:

- Johan Sebastian Castillo Mazo
- Juan Camilo Pavo Quintero

Fecha: 27/02/2026

Asignatura: Estructura de Computadores

Enlace del repositorio en GitHub:

- Github Camilo Pavo
- Github Johan Mazo

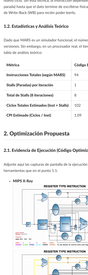
1. Análisis del Código Base

1.1. Evidencia de Ejecución

Adjunte aquí las capturas de pantallas de la ejecución del `programa_base.exe` utilizando los siguientes herramientas de MARS:

- **MIPS X-Ray:**

- **Instruction Counter**



- **Instruction Statistics**

1.2. Identificación de Bloques (Hazards)

Complete la siguiente tabla identificando las instrucciones que causan parada en el pipeline:

Instrucción Ejecutada	Dirección (Hexa)	Tipo de Riesgo	Ciclos de Parada (Con Forwarding)	Ciclos de Parada (Sin Forwarding)
`slt $t4, $t0, $t1`	`0x00c: $t0, $t1, $t4`	Riesgo de Datos (RAW en $t4)	1 (Inmediato)	2
`beq $t4, $zero, 8`	`0x010: $t4, $zero, 8(0x014)`	Riesgo de Control (Salto)	2	2
`mul $t7, $t6, $t5`	`0x018: $t6, $t5, $t7`	Riesgo de Datos (RAW en $t6)	0	2
`add $t9, $t7, $t8`	`0x01c: $t7, $t8, $t9`	Riesgo de Datos (RAW en $t7)	0	2
`sw $t9, $t2(0x100)`	`0x020: $t9, $t2, 0x100`	Riesgo de Datos (RAW en $t9)	1	1
`lw $t3, $t2(0x100)`	Siguiente instrucción (Branch)	Riesgo de Control (Salto)	1	1
`j 0x000`	Siguiente instrucción (Jump)	Riesgo de Control (Salto)	1	1

Aquí podemos ver como el rendimiento del procesador cambia drásticamente dependiendo de si este cuenta o no con la unidad de **Forwarding**. Esta tecnología de hardware permite que el resultado de una operación anterior se pase directamente desde la etapa de Ejecución (EX) a la entrada de la ALU sin el nuevo ciclo. Sin esta técnica, la instrucción dependiente tiene que detenerse ignorando ciclos de parada hasta que el dato termine de escribirse físicamente en el banco de registros durante la etapa final de Write Back (WB) para recién poder leerlo.

1.3. Estadísticas y Análisis Técnico

Dado que MARS es un simulador funcional, el número de instrucciones ejecutadas será igual en ambas versiones. Sin embargo, en un procesador real, el tiempo de ejecución (ciclos) sí varía. Complete la siguiente tabla de análisis técnico:

Métrica	Código Base	Código Optimizado
Instrucciones Totales (según MARS)	94	58
Stalls (Paradas) por Ejecución	10	0
Total de Stalls (8 iteraciones)	80	0
Ciclos Totales Estimados (Inst + Stalls)	102	58
CPI Estimado (Ciclos / Inst)	1.09	1.00

2. Optimización Propuesta

2.1. Evidencia de Ejecución (Código Optimizado)

Adjunte aquí las capturas de pantalla de la ejecución del `programa_optimizado.exe` utilizando los mismas herramientas que en el punto 1.1:

- **MIPS X-Ray**

- **Instruction Counter**

- **Instruction Statistics**

2.2. Código Optimizado

Aquí se detalla la optimización realizada al código original. Si usaste incluso no solo el desenrollado del bucle (loop unrolling), sino también el pre-cálculo del límite de iteraciones y la implementación de aritmética de punteros.

```
.data
    vector_a: .word 7, 2, 5, 4, 3, 6, 1, 8
    vector_c: .space 32
    vector_b: .word 0 : 8
    const_a:  .word 5
    temp:     .word 0

.text
main:
    # -- Inicialización de punteros y constantes --
    la $t0, vector_a      # $t0 = dirección base vector_a A[]
    la $t1, vector_c      # $t1 = puntero actual C[i]
    la $t6, const_a       # $t6 = a = 5
    lw $t6, 0($t6)        # $t6 = a = 5
    la $t2, temp          # $t2 = dirección de elemento
    # -- Pre-cálculo: el límite NUNCA debe leerse dentro del bucle --
    li $t3, 32            # $t3 = 8 * 4 = 32 bytes
    add $t3, $t0, $t3     # $t3 = $t0 + 32 = dirección de parada
    # -- LOOP UNROLLED: procesamos 2 elementos por iteración --
    # Registros: $t0 = puntero A[], $t1 = puntero C[], $t2 = temp
    # $t3 = dirección de elemento 8, $t4/$t5 = temporales
    # Requisito: el vector debe tener numero par de elementos (pasa 8, OK).
loop:
    lw $t4, 0($t0)        # A[i+0] = x
    lw $t5, 4($t0)        # A[i+1] = x
    # -- ALU/FLOATING --
    mul $t7, $t4, $t6     # $t7 = A[i+0] * a
    mul $t4, $t5, $t6     # $t4 = A[i+1] * a
    # -- SW: SIN STALL --
    sw $t7, 0($t1)        # C[i+0] = $t7
    sw $t4, 4($t1)        # C[i+1] = $t4
    # -- ACUMULAMOS --
    sw $t6, 0($t2)        # *temp = a
    lw $t5, 0($t2)        # $t5 = *temp
    # -- CONDICIÓN DE SALIDA --
    sw $t0, $t2, loop     # $t0 == $t2 (siguiente A[i+8]), ¿ya?
    # -- AVANZAMOS --
    addi $t0, $t0, 8      # A += 2 elementos
    addi $t1, $t1, 8      # C += 2 elementos
    j loop                # A[i+8] = dirección de parada

fin:
    li $v0, 10
    syscall
```

2.3. Justificación Técnica de la Mejora

Explique aquí instrucción nueva y por qué colocarla entre el `la a0 a1` y el `mul`, eliminas el riesgo de datos:

La instrucción que insertamos es `lw $t4, 0($t0)`, que carga el elemento A[i+0] del vector. En el código base, el `mul $t7, $t4, $t6` aparecía inmediatamente después de `lw $t4, 0($t0)`, generando un **Load-Use Hazard** de datos (el hilo está disponible al finalizar la etapa MEM del `lw`, pero `mul` lo necesita al inicio de su etapa EX, provocando que el hardware tenga una burbuja automáticamente, prohibido en el ciclo 18).

Para eliminarlo, aplicamos **reordenamiento de instrucciones por software**. Bajamos una instrucción independiente más adelante en el código y la movemos al hueco. La instrucción `lw $t5, 4($t0)` cumple exactamente ese propósito. Al ser independiente, por lo que no genera nuevos dependencias. Al colocarla entre el primer `la a0 a1` y el pipeline tiene el ciclo necesario para que `$t0` esté disponible cuando `mul` lo requiera, cumpliendo el ciclo exacto de latencia.

Como beneficio adicional, el segundo `mul $t4, $t5, $t6` ocupa el hueco de dependencia RAW entre `mul $t7 a0 a1` y `add $t9, $t7, $t8`, ocultando también esa latencia. En esta forma, tres uno dependiendo del elemento-1 (x acceso sincronizado) como reflejo para los hacen del elemento-1.

Finalmente, para potenciar esta optimización, se eliminó el cálculo tradicional de índices en favor de la **aritmética de punteros**. En el código base, recalculaba la dirección de memoria en cada iteración consumo instrucciones adicionales (`sll 1, addu`) para multiplicar el índice y sumarlo a la dirección base. Al pre calcular la dirección límite de memoria fuera del bucle y actualizar los punteros directamente sumando bytes (`addi $t0, $t0, 8`), se ahorró el sobrecosto computacional del contador. Esto redujo drásticamente el número total de instrucciones ejecutadas y dejó el bucle lo suficientemente limpio como para que el desenrollado (loop unrolling) fuera nuevamente efectivo al acercarse la perforación en un ambiente ideal.

3. Comparativa de Resultados

Métrica	Código Base	Código Optimizado	Mejora (%)
Ciclos Totales	102	58	43.1%
Stalls (Paradas)	80	0	100%
CPI	1.09	1.00	8.3%

4. Conclusiones

¿Qué impacto tiene la segmentación en el diseño de software de bajo nivel? ¿Es siempre posible eliminar todos los paradas?

El impacto de la segmentación en el diseño a bajo nivel es irrevocablemente profundo. Tus analizar y optimizar el código en el simulador MARS, entendemos que programar no ensamblador exige conocer la arquitectura física del procesador. La segmentación obliga al programador a estructurar el código estratégicamente para mantener ocupado todas las etapas del pipeline y evitar tiempos muertos. Como ingenieros, a menudo el código base es ineficiente en la resolución lógica del problema, dejando de lado la eficiencia. Sin embargo, el diseño a bajo nivel demuestra que comprender y aprovechar las recursos de hardware disponibles puede resultar en mejoras de rendimiento enormes, reducciones drásticas en los tiempos de ejecución con modificaciones que en principio parecían menores o triviales.

Respecto a la eliminación de las paradas, creemos que no siempre es posible erradicarlas por completo. Aunque en la optimización de este laboratorio logramos un escenario ideal en la práctica esto depende fuertemente de la temática y complejidad del algoritmo. Si un programa tiene una alta densidad de dependencias secuenciales, múltiples saltos condicionales (Control Hazards) o accesos a memoria donde no podemos predecir el futuro, el hardware siempre tendrá que insertar burbujas o ciclos muertos inevitables. Finalmente, el objetivo no es ingeniería de software tampoco debe ser aplicar cualquier técnica posible para reducir el tiempo de ejecución, sino hacer que la lógica sea algorítmicamente correcta.